VALEMONT OS · SETUP

The plan and tutorial for every connection

French version: docs/SETUP_GUIDE.md

For Théo and Pablo. This guide tells you **in what order** to connect each service, **where to click**, **which variable to fill in** and **how to check** that it works. It is based on the project documents (they are the reference): `.env.example`, `docs/DEPLOYMENT.md`, `docs/INTEGRATIONS.md`, `docs/INBOX.md`, `docs/TRACKER.md`, `docs/SECURITY.md`, `docs/ACCEPTANCE.md`. The steps at Google, Vercel, Wise, Stripe, Meta and LinkedIn were checked against their official documentation in November 2026 (links in each section). These interfaces change often: if a menu has a different name, search for the keyword given.

Three rules before you start

1. **A secret (key, token, password) is pasted in one place only: Vercel › Settings › Environment Variables.** Never in a chat (including with Claude), never in an email, never in the Google Sheet, never in the code.
2. **Every account belongs to Valemont** (your Google addresses, your Meta Business Manager, your Vercel account). The Claude account that built the application is never used on your real resources.
3. **After every variable change: Vercel › Deployments › ⋯ › Redeploy.** A new variable only takes effect on the next deployment.

In the application, everything is controlled from **Réglages › Intégrations (Settings › Integrations)**: one row per service (15 in total), with its status (*Connecté* (Connected), *Non configuré* (Not configured), *Erreur* (Error), *À renouveler* (Needs renewal), *Import uniquement* (Import only), *En attente d'approbation* (Awaiting approval), *Démo, simulé* (Demo, simulated)), the **names** of the variables (set / missing, never their value) and the buttons **Tester la connexion** (Test connection), **Synchroniser** (Sync), **Autoriser / Autoriser à nouveau** (Authorize / Authorize again) where they make sense.

---

1. The plan

Tick the boxes as you go. "Who" is a suggestion: what matters is that the person has admin rights on the account concerned.

Phase 0 · Prerequisites and accounts (both of you · 1 h)

- [ ] Choose the application's address, for example `os.<your-domain>` (it is used everywhere: `APP_BASE_URL`).
- [ ] Check that you have: a Google account each (ideally Google Workspace, same domain), access to the GitHub repository `valemontops-boop/valemontops`, a Vercel account, admin access to Wise Business, Stripe, Meta Business (Facebook Page + Instagram professional accounts), the LinkedIn Pages, GA4, Search Console, the website hosting (Netlify or other), and Make, Zapier or n8n if you want the inbox relay.
- [ ] A computer with a terminal and **Node.js 22** installed (https://nodejs.org) for Phase 3 (you only need it once, then again for each update that adds tabs).
- [ ] A shared password manager (1Password, Bitwarden…) to keep a copy of the generated secrets.

Unlocks: nothing yet, but it saves you from getting stuck halfway.

Phase 1 · Google: Sheets, Drive, sign-in (Théo, or whoever holds the Google admin account · 1 h 30)

- [ ] Create the Google Cloud project and enable the 4 APIs (Sheets, Drive, Calendar, Gmail).
- [ ] Create the **service account** and its JSON key → `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`.
- [ ] Create the empty main Google Sheet and share it with the service account → `SHEETS_SPREADSHEET_ID`.
- [ ] (Optional) Each of you creates a private Sheet and shares it → `PRIVATE_SHEET_ID_THEO` / `PRIVATE_SHEET_ID_PABLO`.
- [ ] Share the "Valemont OS" Drive folder (Viewer) and prepare the backup folder (Editor).
- [ ] Share the calendars with the service account.
- [ ] OAuth consent screen + OAuth client → `GOOGLE_OAUTH_CLIENT_ID`, `GOOGLE_OAUTH_CLIENT_SECRET`.

Unlocks: the database (Sheet), Google sign-in for both founders, Files, Calendar.

Phase 2 · Going live on Vercel (Théo or Pablo · 45 min)

- [ ] Import the GitHub repository into Vercel, **Pro** plan, Paris region.

- [ ] Fill in the required variables (list in §2.4), add the domain, redeploy.

Unlocks: the application online in LIVE mode (without the variables, it shows a configuration page, never the demo).

Phase 3 · First start (the person with Node · 30 min, then both of you · 10 min)

- [ ] `npm run setup:sheets` then `npm run doctor:sheets` on the real Sheet: everything ✓.
- [ ] Théo signs in, Pablo signs in from another browser; a non-founder account is refused.
- [ ] Réglages › Calendriers (Settings › Calendars) and Réglages › Espace de travail (Settings › Workspace): paste the calendar IDs and the Drive folder ID.

Unlocks: the whole core application (Ma journée (My Day), CRM, Appels (Calls), Tâches (Tasks), Agenda (Calendar), Fichiers (Files)).

Phase 4 · Money: Wise and Stripe (whoever administers the bank · 1 h)

- [ ] Wise: read-only API token (+ profile ID, + optional SCA key, + webhook).
- [ ] Stripe: read-only restricted key + webhook.

Unlocks: Finances (balances, transactions, sales, fees, payouts), MRR in Finances and the Command Center, USD→EUR rate for supplier payments in dollars. The application can never move money.

Phase 5 · Email (Gmail) and calendar (each for yourself · 10 min each, + possible Google delay)

- [ ] Check that `gmail.send` is on the consent screen, then each of you clicks **Autoriser** (Authorize) on the Gmail row.
- [ ] Check that the Agenda merges the calendars properly (the other founder's personal calendar shows as « Occupé » (Busy)).

Unlocks: one-click email sending from Appels and Croissance › Messages (Growth › Messages) (always after your click). ⏳ With an "External" Google app, consent is requested again often, and publishing to production requires **Google verification** (see §2.6).

Phase 6 · Website and "To answer" inbox (Pablo or whoever manages the websites · 1 h 30)

- [ ] Signed webhook for the FR/ES website forms → `WEBSITE_WEBHOOK_SECRET`.
- [ ] Make / Zapier / n8n relay (comments, LinkedIn notifications…) → `INBOX_WEBHOOK_SECRET`.

Unlocks: every website form becomes an inbound lead + an « À répondre » (To answer) item; comments and messages arrive in Ma journée.

Phase 7 · Social: Instagram, LinkedIn, GA4, Search Console (Théo or Pablo · 2 h + Meta/LinkedIn delays)

- [ ] Instagram: Meta app, token (system user), account ID → automatic statistics.
- [ ] Instagram: comments/messages webhook → "To answer" inbox. ⏳ **Meta App Review** for accounts that have no role in the app.
- [ ] LinkedIn: export imports (.xls Page, .xlsx creator). ⏳ Community Management API: optional **access request**, long and not guaranteed.
- [ ] GA4 and Search Console: CSV exports imported every week.

Unlocks: Croissance (Growth: channels, costs, funnels), Studio contenu (Content Studio: numbers per post), Croissance › SEO.

Phase 8 · Personal tracker (each for yourself · 20 min)

- [ ] URL + secret of your tracking app's webhook, feed token.

Unlocks: each of your activities sent to your personal tracker, and a feed it can read.

Phase 9 · Backups, scheduled jobs, restore test (Théo · 1 h)

- [ ] Backup folder (shared drive, or Apps Script copy if personal Google account) → `BACKUP_DRIVE_FOLDER_ID`.
- [ ] `CRON_SECRET` (both scheduled jobs are already declared in `vercel.json`).
- [ ] Restore drill into a **new** Sheet (never touch the production Sheet).

Unlocks: verified daily copy of the Sheet, Instagram snapshot every 6 h, a plan B if something goes wrong.

Phase 10 · Final checks (both of you · 45 min)

- [ ] Réglages › Intégrations: every configured row is **Connecté** after **Tester la connexion**; GA4 and Search Console = *Import uniquement* (normal); LinkedIn = *En attente d'approbation* (normal).
- [ ] Réglages › Diagnostic (Settings › Diagnostics): health « prête » (ready), backups « vérifiée » (verified).
- [ ] Go through the "Stage 4 gate: live validation" list in `docs/ACCEPTANCE.md` and record the date of each box in `docs/STATUS.md` (nothing is VERIFIED_LIVE before that).

---

2. The tutorial

2.1 Generate a secret (useful everywhere)

Several variables are secrets that **you make up** (not provided by a service). Generate them on your computer, paste them into Vercel and into the password manager:

Need	macOS / Linux (Terminal)	Windows (PowerShell 7)
`SESSION_SECRET`	`openssl rand -base64 48`	`[Convert]::ToBase64String([Security.Cryptography.RandomNumberGenerator]::GetBytes(48))`
all other secrets/tokens (webhooks, cron, tracker)	`openssl rand -hex 32`	`-join ([Security.Cryptography.RandomNumberGenerator]::GetBytes(32) | % { $_.ToString("x2") })`

One secret per use: never reuse the same one for two variables.

2.2 Google Cloud: project, APIs, service account

What it's for: the service account is the "robot" that reads and writes the Google Sheet (the only database), reads the shared Drive folder, reads the calendars and makes the backups.

What you need: your Google account (preferably Workspace), 30 minutes.

Steps

1. Go to https://console.cloud.google.com › project picker › **New Project** › name "Valemont OS". (Docs: https://developers.google.com/workspace/guides/create-project)
2. **APIs & Services › Library**: enable **Google Sheets API**, **Google Drive API**, **Google Calendar API**, **Gmail API**. (Docs: https://developers.google.com/workspace/guides/enable-apis)
3. **IAM & Admin › Service Accounts › Create service account**: name `valemont-os`, **no role** (its rights come only from the shares you make). Note its address: `valemont-os@<project>.iam.gserviceaccount.com`. (Docs: https://cloud.google.com/iam/docs/service-accounts-create)
4. Open the service account › **Keys** tab › **Add key › Create new key › JSON**. A file downloads. (Docs: https://cloud.google.com/iam/docs/keys-create-delete)
5. Encode it in base64 and copy it to the clipboard:
   - macOS: `base64 -i key.json | pbcopy`
   - Linux: `base64 -w0 key.json`

    - Windows: `[Convert]::ToBase64String([IO.File]::ReadAllBytes("key.json")) | Set-Clipboard`
6. Paste the value into Vercel (Phase 2) as `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`, then **delete the `key.json` file** (and empty the trash).

Variables: `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`.

Common problems

- *"Service account key creation is disabled"*: recent Google Cloud organizations block keys by default (policy `iam.disableServiceAccountKeyCreation`). The Workspace admin must lift this policy **for this project only** (IAM › Organization policies), create the key, then put the policy back.
- Lost or leaked key: create a new key, replace the variable, redeploy, **then** delete the old key.

2.3 Google Sheets (database) and private Sheets

What it's for: everything you enter is read and written here. The private Sheets hold each founder's private tasks and applications, invisible to the other.

Steps

1. On https://sheets.google.com, create an **empty** spreadsheet "Valemont OS" (the application never creates it).
2. **Share** › paste the service account's address › role **Editor** › untick "Notify people".
3. Copy the ID from the URL: `https://docs.google.com/spreadsheets/d/<THIS-ID>/edit` → `SHEETS_SPREADSHEET_ID`.
4. (Optional) **Each of you** creates, **from your own account**, a spreadsheet "Valemont OS – privé ", shares it with the service account as **Editor** (and with no one else) → `PRIVATE_SHEET_ID_THEO`, `PRIVATE_SHEET_ID_PABLO`.
5. The tabs and headers are created in Phase 3 by `npm run setup:sheets`.

Variables: `SHEETS_SPREADSHEET_ID` (required), `PRIVATE_SHEET_ID_THEO`, `PRIVATE_SHEET_ID_PABLO` (optional).

Check: Réglages › Intégrations › Google Sheets › **Tester la connexion**; `npm run doctor:sheets` all ✓.

Common problems

- *Permission denied*: the share was made with a different address, or as Viewer instead of Editor.
- Don't rename or delete tabs/columns by hand: the application finds columns by their name. Adding formatting rows or formulas in the Sheet breaks reading (Diagnostic reports it).

2.4 Google Sign-In (sign-in for both founders)

What it's for: only your two Google addresses can sign in. The same OAuth client is also used for the Drive (`drive.file`) and Gmail (`gmail.send`) permissions, requested later, one at a time.

Steps (the menu is now called **Google Auth Platform** in the console)

1. **Google Auth Platform › Branding**: name "Valemont OS", support email = a founder's address, authorized domain = your domain. (Docs: https://developers.google.com/workspace/guides/configure-oauth-consent)
2. **Audience**:
   - if you are **both in the same Google Workspace**: type **Internal** (the simplest: no verification, no test users);
   - otherwise: type **External**, status **Testing**, and add both your addresses under **Test users**.
3. **Data access › Add or remove scopes**: `openid` , `.../auth/userinfo.email` , `.../auth/userinfo.profile` , `.../auth/drive.file` , `.../auth/gmail.send` . (The last two are only requested when you use them: this is incremental authorization, https://developers.google.com/identity/protocols/oauth2/web-server#incrementalAuth)
4. **Clients › Create client** › type **Web application** › name "Valemont OS". **Authorized redirect URIs** (replace with your address, no trailing `/`):
   - `https://os.<domain>/api/auth/google/callback`
   - `https://os.<domain>/api/auth/google/drive/callback`
   - `https://os.<domain>/api/auth/google/gmail/callback`
5. Copy the **Client ID** → `GOOGLE_OAUTH_CLIENT_ID` and the **Client secret** → `GOOGLE_OAUTH_CLIENT_SECRET` (straight into Vercel).

Variables: `GOOGLE_OAUTH_CLIENT_ID` , `GOOGLE_OAUTH_CLIENT_SECRET` , `FOUNDER_THEO_EMAIL` , `FOUNDER_PABLO_EMAIL` , `FOUNDER_THEO_TIMEZONE` (`Europe/Paris`), `FOUNDER_PABLO_TIMEZONE` (`Europe/Madrid`), `APP_BASE_URL` .

Check: open the application, « Se connecter avec Google » (Sign in with Google): Théo and Pablo get in, a third address is refused.

Common problems

- `redirect_uri_mismatch` : the registered URI doesn't exactly match `APP_BASE_URL` (https, no trailing `/` , same subdomain). If you change domain, add the 3 new URIs.
- *"Access blocked: app has not completed the Google verification process"*: in External/Testing, the address is not among the test users.

2.5 Google Drive and Google Calendar

What it's for: Fichiers (Files) lists the "Valemont OS" folder; uploading a file uses **your** `drive.file` permission (the file belongs to you). Agenda (Calendar) merges the shared "Valemont" calendar and

your calendars; the other founder's personal calendar only shows as « Occupé » (Busy). Founder meetings are written to the shared calendar.

Drive steps

1. In Google Drive, "Valemont OS" folder › **Share** › service account as **Viewer**.
2. Copy the ID from the URL `https://drive.google.com/drive/folders/<ID>` and paste it into the application: **Réglages › Espace de travail › Identifiant du dossier Drive** (Settings › Workspace › Drive folder ID) (not an environment variable).

Calendar steps (Docs: https://support.google.com/calendar/answer/37082)

1. Google Calendar › ⚙ **Settings** › choose the shared "Valemont" calendar › **Share with specific people or groups** › add the service account with **Make changes to events**.
2. Each of you, for your personal calendar: same path, but with **See only free/busy (hide details)**.
3. In each calendar › **Integrate calendar › Calendar ID**: copy it into **Réglages › Calendriers**.

Variables: none new (`GOOGLE_SERVICE_ACCOUNT_KEY_BASE64` is enough).

Check: Réglages › Intégrations › Google Drive and Google Calendar › **Tester la connexion**; open Fichiers and Agenda. First file upload: Google asks you for the Drive permission once (it lasts about 1 h, then one click on **Autoriser à nouveau**).

Common problems: empty calendar = wrong ID or forgotten share; invisible file = it isn't in the shared folder.

2.6 Gmail (send only)

What it's for: the emails prepared by the application (after a call, follow-ups, Croissance › Messages) are sent from **your** Gmail **when you click**. The application never reads your inbox. Without Gmail connected, it opens a pre-filled Gmail draft: nothing is blocked.

Steps

1. Check (Phase 1, step 3 of §2.4) that the Gmail API is enabled, that `gmail.send` is in Data access and that the URI `/api/auth/google/gmail/callback` is registered.
2. Each of you, in the application: **Réglages › Intégrations › Gmail › Autoriser** (or « Connecter Gmail » (Connect Gmail) in Appels). Google shows the consent screen: accept.

Variables: none new (same OAuth client).

Check: the Gmail row shows the authorization as active in this browser. There is **no** « Tester la connexion » for Gmail (a test would send a real email): send yourself a test email from a contact record to your address.

⏳ **Google delays and limits** (Docs: https://developers.google.com/workspace/gmail/api/auth/scopes, https://support.google.com/cloud/answer/13463073)

- `gmail.send` is a **sensitive** scope.
- **Internal** app (Workspace): no verification, nothing to wait for. This is the recommended choice.
- **External app in Testing**: works for your two addresses ("unverified app" screen to accept), but Google asks for consent again often (at least every 7 days).
- Moving an **External app to production** with `gmail.send` requires **Google verification** (demo video, privacy policy, verified domain): allow **several days to a few weeks**.

2.7 Vercel: going live

What it's for: hosting the application, keeping the secrets, running the two scheduled jobs.

Steps

1. https://vercel.com › **Add New** › **Project** › **Import Git Repository** › authorize Vercel's GitHub app on `valemontops-boop/valemontops` › Framework: Next.js (detected), default build. (Docs: https://vercel.com/docs/getting-started-with-vercel/import)
2. **Pro plan**: the Hobby plan is restricted to personal non-commercial use and doesn't allow a job every 6 h. A single paying member is enough. (Docs: https://vercel.com/docs/limits/fair-use-guidelines, https://vercel.com/docs/cron-jobs/usage-and-pricing)
3. **Settings** › **Functions**: region **Paris (cdg1)**; leave the Node.js runtime and Fluid Compute (default).
4. **Settings** › **Environment Variables**: add each variable, environment **Production**, and tick **Sensitive** for every secret (the value then becomes unreadable, even for you). (Docs: https://vercel.com/docs/environment-variables). Minimum to start: `APP_MODE=live`, `SESSION_SECRET`, `APP_BASE_URL`, `FOUNDER_THEO_EMAIL`, `FOUNDER_PABLO_EMAIL`, `FOUNDER_THEO_TIMEZONE`, `FOUNDER_PABLO_TIMEZONE`, `GOOGLE_OAUTH_CLIENT_ID`, `GOOGLE_OAUTH_CLIENT_SECRET`, `SHEETS_SPREADSHEET_ID`, `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`, `CRON_SECRET`, `LOG_LEVEL=info`.
5. **Settings** › **Domains** › add `os.<domain>` › create the DNS record shown (CNAME) at your registrar › wait for the certificate. Set `APP_BASE_URL=https://os.<domain>` (no trailing `/`). (Docs: https://vercel.com/docs/domains/working-with-domains/add-a-domain)
6. **Deployments** › ⋯ › **Redeploy**.
7. Scheduled jobs: nothing to create, `vercel.json` declares the daily backup (02:17 UTC) and the Instagram snapshot (every 6 h). Vercel calls them with `Authorization: Bearer <CRON_SECRET>`. They only run on the production deployment. Visible in **Settings** › **Cron Jobs**. (Docs: https://vercel.com/docs/cron-jobs/manage-cron-jobs#securing-cron-jobs)

Never put in production: `SHEETS_API_BASE_URL`, `GOOGLE_API_BASE_URL`, `WISE_API_BASE_URL`, `STRIPE_API_BASE_URL`, `DEMO_BACKEND`, `FAKE_SHEETS_URL`, `FAKE_SHEETS_SPREADSHEET_ID`,

`DEMO_ACCESS_CODE` (test and demo tools). `INSTAGRAM_API_BASE_URL` only in the Instagram Login case (§2.12).

Check: `https://os.<domain>/api/health` answers `{"status":"ok"}` or `"degraded"` (`fail` = a required variable is missing or the Sheet is unreachable).

Common problems: « configuration requise » (configuration required) page = a required variable is missing (its name is shown); variable added but nothing changes = not redeployed.

2.8 First start (Phase 3)

These commands prepare the Sheet. You run them **once** from a computer, with your own access.

1. Get the code: `git clone https://github.com/valemontops-boop/valemontops.git` then `cd valemontops` and `npm ci`.
2. Copy `.env.example` to `.env.local` and fill in at least `APP_MODE=live`, `SHEETS_SPREADSHEET_ID`, `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64` (the same values as in Vercel). This file is never pushed to GitHub.
3. `npm run setup:sheets`: creates the tabs and headers, freezes the headers, adds a warning-only protection. Safe to run again: it never deletes anything. Run it again after each update that adds columns.
4. `npm run doctor:sheets`: every check must be ✓ (sharing, headers, duplicates, schema).
5. **Delete** `.env.local` when you're done (or keep it encrypted): it contains the service account key.
6. In the application: Théo signs in, Pablo signs in from another browser. Fill in Réglages › Calendriers and Réglages › Espace de travail. Open **Réglages › Diagnostic**: health « prête ».

2.9 Wise (bank, read-only)

What it's for: the Finances page and the Command Center's cash position: balances, statements, live transactions via webhook. The same read-only token also provides **Wise's USD→EUR rates** used for **supplier payments in dollars** (feature being added). **The token can never move money.**

What you need: admin access to the **Wise Business** account, your phone (two-factor authentication).

Steps (Docs: https://docs.wise.com/guides/developer/auth-and-security/personal-api-token)

1. wise.com › Business account › **Settings › API tokens** (depending on the version: *Connect and manage apps › API tokens* or *Integrations and Tools › API tokens*) › **Add new token** › name "Valemont OS" › access **Read only** › confirm two-factor authentication. The token is shown **only once**: paste it straight into Vercel → `WISE_API_TOKEN`.
2. (Optional) `WISE_PROFILE_ID`: without it, the application takes the token's first Business profile. To find it, in a terminal (the token is not shown on screen): `read -s WISE_TOKEN &&`

`curl -s -H "Authorization: Bearer $WISE_TOKEN" https://api.wise.com/v2/profiles` → the `id` of the profile of type `BUSINESS`.

3. (Optional) **SCA key** to read statements when Wise requires strong authentication:
   - `openssl genrsa -out wise-private.pem 2048` then `openssl rsa -pubout -in wise-private.pem -out wise-public.pem`;
   - Wise › API tokens › **Manage public keys** › add the contents of `wise-public.pem`;
   - `base64 -i wise-private.pem` (macOS) → `WISE_SCA_PRIVATE_KEY_BASE64`, then delete the `.pem` files.
   - Warning: in the EU/UK (PSD2), Wise may refuse statement access through a personal token, even with the key. In that case balances and the webhook still work, and the history comes from a **Wise statement CSV export**: Finances › **Importer un CSV Wise** (Import a Wise CSV).
4. **Transactions webhook** (Docs: https://docs.wise.com/guides/developer/webhooks/subscribe-to-webhooks):
   - URL: `https://os.<domain>/api/webhooks/wise`, event `balances#update`;
   - in Wise › Settings › **Webhooks** (developer tools) if the menu exists on your account; otherwise via the API: `curl -X POST -H "Authorization: Bearer $WISE_TOKEN" -H "Content-Type: application/json" https://api.wise.com/v3/profiles/<PROFILE_ID>/subscriptions -d '{"name":"Valemont OS","trigger_on":"balances#update","delivery":{"version":"2.0.0","url":"https://os.<domain>/api/webhooks/wise"}}'` (if Wise refuses with a read-only token, create the subscription from the interface, or skip this step: the **Synchroniser** button catches up on transactions);
   - copy Wise's **production public key** for webhooks ("Event handling" page of the docs: https://docs.wise.com/guides/developer/webhooks/event-handling), save it to `wise-webhook.pem`, `base64 -i wise-webhook.pem` → `WISE_WEBHOOK_PUBLIC_KEY_BASE64`.

Variables: `WISE_API_TOKEN` (required for Wise), `WISE_PROFILE_ID`, `WISE_SCA_PRIVATE_KEY_BASE64`, `WISE_WEBHOOK_PUBLIC_KEY_BASE64` (optional).

Check: Réglages › Intégrations › Wise › **Tester la connexion** (reads balances) then **Synchroniser** (statements). Balances appear in Finances. A credit automatically classified as revenue stays « à vérifier » (to review) until a founder has confirmed it.

Common problems: 401 = token revoked or pasted wrong (extra space); "SCA required" = SCA key missing or rejected → CSV import; wrong profile = set `WISE_PROFILE_ID`.

2.10 Stripe (read-only)

What it's for: Stripe balance, sales, fees, payouts, MRR (Finances and Command Center). The key can neither charge nor refund. Stripe → Wise payouts are internal transfers: a sale is never counted twice.

Steps (Docs: https://docs.stripe.com/keys/restricted-api-keys, https://docs.stripe.com/webhooks)

1. Stripe Dashboard (**live** mode) › **Developers › API keys › Create restricted key** › name "Valemont OS". Set **Read** on: *Balance*, *Balance transactions*, *Subscriptions*, *Customers*. Everything else: **None**. Copy the key (`rk_live_…`) into Vercel → `STRIPE_API_KEY` .
2. **Developers › Webhooks** (in Workbench: *Event destinations*) › **Add destination** › URL `https://os.<domain>/api/webhooks/stripe` › events: `charge.*` , `payout.*` , `customer.subscription.*` , `invoice.paid` , `balance.available` .
3. Open the destination › **Signing secret** › Reveal (`whsec_…`) → `STRIPE_WEBHOOK_SECRET` .

Variables: `STRIPE_API_KEY` (required for Stripe), `STRIPE_WEBHOOK_SECRET` (optional but recommended).

Check: Tester la connexion (reads the balance), **Synchroniser**; in Stripe, *Send test event* on the destination must answer 200.

Common problems: 403 "missing permission" = a resource isn't set to Read (the message says which one); 400 on the webhook = secret from another destination or from test mode.

2.11 Website: forms → signed webhook

What it's for: every form (contact, lead, booking) on the FR/ES websites creates an inbound lead (owner Théo for FR, Pablo for ES) and an « À répondre » item. An unsubscribed contact is blocked: nothing is created.

Steps

1. Generate `WEBSITE_WEBHOOK_SECRET` (`openssl rand -hex 32` , 32 characters minimum) and put it in Vercel.
2. Put **the same secret** in the website hosting (Netlify › Site configuration › Environment variables) along with `VALEMONT_WEBHOOK_URL=https://os.<domain>/api/webhooks/website` .
3. Netlify Forms: Netlify's outgoing webhook can't sign, so you need a **submission-created** function (ready-to-copy code in `docs/INTEGRATIONS.md` › "Netlify Forms"). Docs: https://docs.netlify.com/functions/trigger-on-events/. Other host: any server-side code (Cloudflare Worker, Apps Script…) that signs the same way (`docs/INTEGRATIONS.md` › "Generic form handler"). **The secret must never be in browser code.**
4. In the form: honeypot field `bot-field` , hidden fields `lang` , `page` , UTM, `consent_version` , and a consent checkbox whose sentence is sent as `consent_text` .

Variables: `WEBSITE_WEBHOOK_SECRET` (+ `VALEMONT_WEBHOOK_URL` on the website side, not in Vercel).

Check: Tester la connexion (checks that the secret is present and long enough), then send a real test form: it appears in Ma journée › À répondre (My Day › To answer).

Common problems: 401 `stale_timestamp` = server clock off by more than 5 min; 401 `bad_signature` = different secrets or signature computed on reformatted JSON (you must sign the exact text sent).

2.12 "To answer" inbox: Make / Zapier / n8n relay and Instagram

What it's for: gathering what's waiting for a reply (comments, DMs, LinkedIn notifications…) with a suggested reply in your language. **Nothing is ever sent automatically**: « Copier une réponse » (Copy a reply) opens the conversation and you paste it yourself. Flagged email/WhatsApp replies and website leads already arrive without connecting anything.

A. Relay (Make, Zapier or n8n)

1. Generate `INBOX_WEBHOOK_SECRET` and put it in Vercel **and** in the relay (variable / protected connection).
2. Trigger of your choice: "Instagram for Business › Watch comments", the relay's mailbox (Make *Mailhook*, *Email by Zapier*) fed by a **Gmail filter** that forwards LinkedIn notifications, etc.
3. Action: HTTP POST to `https://os.<domain>/api/inbox/webhook`, JSON body (format in `docs/INBOX.md`), header `X-Valemont-Signature: sha256=<HMAC-SHA256 of the exact body with the secret>`: Make = function `sha256(body; "hex"; secret)` in HMAC mode; Zapier = *Code by Zapier* step; n8n = *Crypto* node (HMAC, SHA256, hex). `external_id` must be stable (it's the duplicate guard).

B. Instagram via the Meta app (see also §2.13 for the app itself)

1. In the Meta app › **Webhooks** (or Instagram › *API setup* › *Configure webhooks*): callback URL `https://os.<domain>/api/inbox/meta`, verify token = a value you generate → `META_VERIFY_TOKEN`. (Docs: https://developers.facebook.com/docs/instagram-platform/webhooks)
2. Subscribe the `comments` and `messages` fields for both accounts (Valemont, 30/30).
3. **App settings › Basic › App secret** → `META_APP_SECRET`.
4. (Optional) Name the accounts: setting `instagram_account_labels` (see `docs/INBOX.md`).
5. ⏳ Permissions `instagram_business_manage_comments`, `instagram_business_manage_messages` (or the equivalents `instagram_manage_comments` / `instagram_manage_messages`): to receive events from accounts that have no role in the app, Meta requires **App Review** (a few days to a few weeks, demo video, privacy policy). In the meantime, use the Make "Watch comments" relay. (Docs: https://developers.facebook.com/docs/resp-plat-initiatives/individual-processes/app-review)

Variables: `INBOX_WEBHOOK_SECRET`, `META_APP_SECRET`, `META_VERIFY_TOKEN`.

Check: Réglages › Intégrations › Boîte À répondre (To-answer inbox) › **Tester la connexion**; then a test comment on Instagram (or a call from the relay) appears in Ma journée › À répondre.

Common problems: Meta can't "verify and save" the URL = `META_VERIFY_TOKEN` different or not redeployed; 401 from the relay = signature computed on a different text than the body sent.

2.13 Instagram: statistics (read-only) and snapshot every 6 h

What it's for: followers, views, reach, profile visits, website clicks, and numbers for each post/Reel at 24 h, 72 h, 7 d, 30 d, stories every 6 h (Studio contenu, Croissance). Without a token: manual entry, marked « saisi à la main » (entered by hand).

What you need: each Instagram account as a **professional account** (Business or Creator) **linked to a Facebook Page** (Docs: https://developers.facebook.com/docs/instagram-platform/overview), all within **Valemont's Meta business portfolio**.

Steps (recommended method: Facebook Login + system user, non-expiring token)

1. https://developers.facebook.com › **My Apps › Create App** › type Business, attached to the Valemont portfolio (never a third party's personal account). Add the **Instagram** product (API with Facebook Login). (Docs: https://developers.facebook.com/docs/instagram-platform/instagram-api-with-facebook-login/)
2. **Business settings** (business.facebook.com/settings) › **Users › System users › Add** (admin role) › **Assign assets**: the app, the Page and the Instagram accounts. (Docs: https://developers.facebook.com/docs/marketing-api/system-users/)
3. **Generate token** › Valemont OS app › expiration **Never** › permissions `instagram_basic`, `instagram_manage_insights`, `pages_show_list`, `pages_read_engagement` → `INSTAGRAM_ACCESS_TOKEN`.
4. Instagram account ID: in the Graph API Explorer, `me/accounts?fields=instagram_business_account` → the field `instagram_business_account.id` → `INSTAGRAM_BUSINESS_ACCOUNT_ID`.
5. One account: `INSTAGRAM_ACCOUNT_KEY=ig-valemont` (or `ig-thirty`). **Two accounts**: instead, `INSTAGRAM_ACCOUNTS_JSON` (exact format in `.env.example`).
6. Standard Access on your own accounts: **no App Review needed** for statistics.

Fallback method (Instagram Login, 60-day token): permissions `instagram_business_basic` + `instagram_business_manage_insights`, `INSTAGRAM_API_BASE_URL=https://graph.instagram.com/v21.0`, `INSTAGRAM_TOKEN_EXPIRES_AT=YYYY-MM-DD`. The application shows the days remaining and creates a task 14 days before; renewal: https://developers.facebook.com/docs/instagram-platform/reference/refresh_access_token, then replace the variable and redeploy.

Snapshot every 6 h: already scheduled by Vercel Cron (`CRON_SECRET`). `SNAPSHOT_JOB_SECRET` / `SNAPSHOT_JOB_TOKEN` are only needed if another scheduler calls it (GitHub Actions, Apps Script…).

Check: Tester la connexion (reads the profile), **Synchroniser**; the « Tâche de capture » (Snapshot job) row shows the last run; Studio contenu shows the numbers.

Common problems: no `instagram_business_account` = account not professional or not linked to the Page; story statistics "not available" under 5 viewers (this is normal, never 0); demographics missing under 100 followers.

2.14 LinkedIn (imports, and API later)

What it's for: impressions, followers and countries for the Valemont / 30/30 Pages and your creator profiles, in Croissance. LinkedIn prospecting stays manual (log + opening the profile): no scraping, no automatic messages.

Now: the exports

1. LinkedIn Page (as admin) › **Analytics** › tab (Content, Visitors, Followers) › **Export** › period › **.xls** file.
2. Creator profile: your profile's **Analytics** › **Export** › **.xlsx** file.
3. In the application: Croissance › **Ajouter des chiffres** (Add numbers) › **Importer un export LinkedIn** (Import a LinkedIn export). Re-importing the same file adds no duplicates. Once a week is enough.

Later (⏳ optional): Community Management API

- Create a **new** dedicated LinkedIn Developers app (no other product), linked to Valemont's **verified Page**, request *Development tier* access, then *Standard tier* with a video of each use case. Long review, possible refusal (you then need a new app). The adapter will only be built after approval. (Docs: https://learn.microsoft.com/en-us/linkedin/marketing/community-management/community-management-overview, https://learn.microsoft.com/en-us/linkedin/marketing/increasing-access)

Variables: none. **Expected status:** *En attente d'approbation* (normal).

2.15 GA4 and Search Console (CSV imports)

What it's for: website visits and leads by source (including AI engines) in Croissance; queries and pages (clicks, impressions, position) in Croissance › SEO and the Studio contenu optimizer.

GA4 (Docs: https://support.google.com/analytics/answer/9317657)

1. analytics.google.com › **Reports** › Acquisition › Traffic acquisition › period (e.g. last 7 days).
2. **Share this report** (top right) › **Download File** › **CSV**.
3. Croissance › **Ajouter des chiffres** › import the GA4 CSV.

Search Console (Docs: https://support.google.com/webmasters/answer/7576553)

1. search.google.com/search-console › website property › **Performance** › period.

2. **Export** › **Download CSV**.
3. Croissance › SEO › **Ajouter des chiffres Search Console** (Add Search Console numbers).

Variables: none. **Expected status:** *Import uniquement* with the date of the last import. Suggested routine: on Monday, both exports.

2.16 Mon tracker (My tracker: your personal tracking app)

What it's for: each of your activities (calls, emails, visits…) is sent to your personal tracker, signed; your tracker can also read your activities through a feed. LIVE only. Each of you configures **your own**: the other founder doesn't see your variables.

Steps

1. In your tracking app, find its **incoming webhook URL** (https required) → `TRACKER_WEBHOOK_URL_THEO` (or `_PABLO`).
2. Generate a secret (16+ characters) → `TRACKER_WEBHOOK_SECRET_THEO` (or `_PABLO`), and give it to your tracker so it can verify the `X-Valemont-Signature` header (verification code in `docs/TRACKER.md`).
3. (Optional) Feed: generate a token of 24+ characters → `TRACKER_FEED_TOKEN_THEO` (or `_PABLO`); your tracker calls `GET https://os.<domain>/api/tracker/feed?from=YYYY-MM-DD&to=YYYY-MM-DD` with `Authorization: Bearer <token>`.
4. (Optional) `TRACKER_DAILY_SUMMARY_THEO=1` to also send the day's summary when the daily review is saved.

Check: Réglages › **Mon tracker** › **Envoyer un événement de test** (Send a test event) (or **Tester la connexion** in Intégrations); « Envoyer cette période » (Send this period) fills a gap without duplicates.

Common problems: `http://` URL refused; the tracker must answer 2xx in under 5 s.

2.17 Backups, scheduled jobs, restore

What it's for: a daily Google Sheet copy, verified, kept 30 days; a tested restore.

Steps

1. Backup folder:
   - **Google Workspace**: create a **shared drive** "Valemont OS – sauvegardes", add the service account as **Content manager**;
   - **personal Google account**: a service account has no storage space. Create a normal folder, share it as Editor, and make the copy with your own small **Apps Script** (code and daily trigger in `docs/DEPLOYMENT.md` › 6. Backups).
2. Folder ID (URL) → `BACKUP_DRIVE_FOLDER_ID`. `BACKUP_RETENTION_DAYS=30` (1 to 365).

3. `CRON_SECRET` (24+ characters) in Vercel: Vercel Cron runs the backup at 02:17 UTC. `BACKUP_JOB_SECRET` / `BACKUP_JOB_TOKEN` only for another scheduler.
4. **Restore drill** (once, then every quarter): create an **empty** Sheet "Valemont OS (restauré YYYY-MM-DD)", share it with the service account, then from the Phase 3 computer: `npm run restore:sheets -- --from-sheet <backup copy id> --to <new Sheet id>` → ✓, then `SHEETS_SPREADSHEET_ID=<new id> npm run doctor:sheets` → ✓. **Don't switch production over** for a drill. The script refuses the production Sheet and any non-empty Sheet.
5. (Optional) An uptime monitor (UptimeRobot, Better Stack…) on `https://os.<domain>/api/health`, at most once a minute.

Check: Réglages › Diagnostic › **Sauvegardes** (Backups): "Valemont OS backup <today's date>" present and **vérifiée** (verified); **Vérifier** (Verify) button to compare with the current database.

Common problems: `storageQuotaExceeded` = personal account without a shared drive → Apps Script solution.

2.18 Final checks (Phase 10)

1. Réglages › Intégrations: click **Tester la connexion** on every configured row → **Connecté**.
2. Réglages › Diagnostic: health « prête », backup verified.
3. Go through the "Stage 4 gate: live validation" checklist in `docs/ACCEPTANCE.md` together (both sign-ins, a call seen by the other founder in under a minute, edit conflict, `+33…` intact in the Sheet, calendar, files, LinkedIn imports, backup, restore, `/api/health`, security headers).
4. Record the dated result in `docs/STATUS.md`. An integration is only **VERIFIED_LIVE** after this real test.

---

3. All the variables

Paste into **Vercel › Settings › Environment Variables** (Production, *Sensitive* for secrets), then **Redeploy**. "You" = a value you generate yourselves (§2.1).

Variable	Required?	Where to find it	Integration
`APP_MODE`	Required (`live`)	You	Core
`SESSION_SECRET`	Required	You (`openssl rand -base64 48`)	Core
`APP_BASE_URL`	Required	Your domain, `https://…` with no trailing `/`	Core, Sign-In, webhooks
`SYNC_POLL_SECONDS`	Optional (30)	30 to 60	Core
`LOG_LEVEL`	Optional (`info`)	`debug` / `info` / `warn` / `error` / `silent`	Logs
`FOUNDER_THEO_EMAIL` , `FOUNDER_PABLO_EMAIL`	Required	Your Google addresses	Sign-In
`FOUNDER_THEO_TIMEZONE` , `FOUNDER_PABLO_TIMEZONE`	Required	`Europe/Paris` , `Europe/Madrid`	Core
`GOOGLE_OAUTH_CLIENT_ID` , `GOOGLE_OAUTH_CLIENT_SECRET`	Required	Google Auth Platform › Clients	Sign-In, Drive, Gmail
`SHEETS_SPREADSHEET_ID`	Required	Sheet URL	Google Sheets
`GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`	Required	Service account JSON key, in base64	Sheets, Drive, Calendar, backups
`PRIVATE_SHEET_ID_THEO` , `PRIVATE_SHEET_ID_PABLO`	Optional	URL of each founder's private Sheet	Google Sheets (private)
`WISE_API_TOKEN`	Optional (required for Wise)	Wise › Settings › API tokens (Read only)	Wise
`WISE_PROFILE_ID`	Optional	`GET /v2/profiles` (BUSINESS profile)	Wise
`WISE_SCA_PRIVATE_KEY_BASE64`	Optional	Your RSA key (public part uploaded to Wise), in base64	Wise
`WISE_WEBHOOK_PUBLIC_KEY_BASE64`	Optional	Wise webhooks production public key (Wise docs), in base64	Wise
`STRIPE_API_KEY`	Optional (required for Stripe)	Stripe › Developers › API keys › restricted key	Stripe
`STRIPE_WEBHOOK_SECRET`	Optional	Stripe › Webhooks › destination › secret `whsec_…`	Stripe
`WEBSITE_WEBHOOK_SECRET`	Optional (required for forms)	You (32+ chars), also on the website side	Website forms

Variable	Required?	Where to find it	Integration
`INBOX_WEBHOOK_SECRET`	Optional (required for the relay)	You, also in Make/Zapier/n8n	To-answer inbox
`META_APP_SECRET`	Optional	Meta app › Settings › Basic	Inbox (Instagram)
`META_VERIFY_TOKEN`	Optional	You, also in the Meta Webhooks config	Inbox (Instagram)
`INSTAGRAM_BUSINESS_ACCOUNT_ID`	Optional (required for 1 account)	Graph API Explorer	Instagram
`INSTAGRAM_ACCESS_TOKEN`	Optional (required for 1 account)	Meta system user (or Instagram Login)	Instagram
`INSTAGRAM_ACCOUNT_KEY`	Optional	`ig-valemont` (default) or `ig-thirty`	Instagram
`INSTAGRAM_TOKEN_EXPIRES_AT`	Optional	Expiry date of a 60-day token	Instagram
`INSTAGRAM_ACCOUNTS_JSON`	Optional (instead of the 4 above)	You (JSON list, see `.env.example`)	Instagram
`INSTAGRAM_API_BASE_URL`	Optional	Only `https://graph.instagram.com/v21.0` with Instagram Login	Instagram
`SNAPSHOT_JOB_SECRET`, `SNAPSHOT_JOB_TOKEN`	Optional	You (if a scheduler other than Vercel)	Snapshot job
`TRACKER_WEBHOOK_URL_THEO` / `_PABLO`	Optional	Your tracking app (https)	My tracker
`TRACKER_WEBHOOK_SECRET_THEO` / `_PABLO`	Optional (required with the URL)	You (16+ chars)	My tracker
`TRACKER_FEED_TOKEN_THEO` / `_PABLO`	Optional	You (24+ chars)	My tracker
`TRACKER_DAILY_SUMMARY_THEO` / `_PABLO`	Optional	`1` to enable	My tracker
`BACKUP_DRIVE_FOLDER_ID`	Optional (recommended)	URL of the backup folder / shared drive	Backups
`BACKUP_RETENTION_DAYS`	Optional (30)	1 to 365	Backups
`BACKUP_JOB_SECRET`, `BACKUP_JOB_TOKEN`	Optional	You (if a scheduler other than Vercel)	Backups
`CRON_SECRET`	Optional (recommended on Vercel)	You (24+ chars)	Backups + snapshot

Variable	Required?	Where to find it	Integration
`SHEETS_API_BASE_URL`, `GOOGLE_API_BASE_URL`, `WISE_API_BASE_URL`, `STRIPE_API_BASE_URL`, `DEMO_BACKEND`, `FAKE_SHEETS_URL`, `FAKE_SHEETS_SPREADSHEET_ID`, `DEMO_ACCESS_CODE`	**Never in production**	Tests and demo only	—

On the website side (not in Valemont OS's Vercel): `WEBSITE_WEBHOOK_SECRET` (same value) and `VALEMONT_WEBHOOK_URL`. Calendar IDs and the Drive folder ID are entered **in the application** (Réglages › Calendriers, Réglages › Espace de travail), not as variables.

---

4. Security reminders

- **Secrets live only in Vercel** (ticked *Sensitive*) and in your password manager. Never in a chat, an email, the Sheet, a ticket, a screenshot or the code. The application never displays them: Réglages › Intégrations only shows "set / missing". Logs mask keys, emails and phone numbers.
- **Least privilege everywhere:** service account with no role (only the shares it needs), Drive folder as Viewer, personal calendars as "free/busy only", Wise **read-only**, Stripe **restricted Read key**, Instagram statistics/read, Gmail **send only**, Drive as `drive.file` (only the files created by the app).
- **What the application never does:**
  - send an email, a WhatsApp, an Instagram or LinkedIn reply without **your click**;
  - read your Gmail inbox;
  - move money, pay, refund (Wise and Stripe are read-only);
  - contact an unsubscribed contact (blocked on the server, for both of you);
  - show one founder the other's private data (private tasks, personal calendar details);
  - use the demo instead of the real data: without credentials, LIVE mode shows a configuration error.
- **Rotation:**
  - leak or doubt about a secret → regenerate it at the provider **and** in Vercel at the same time, redeploy;
  - Google key → new key, variable, redeploy, **then** delete the old one;
  - changing `SESSION_SECRET` = everyone is signed out (this is how you revoke a stolen session);
  - 60-day Instagram token: the application creates a task 14 days before expiry;
  - a contractor leaves or you change computer: redo the tokens they may have seen.
- **One owner per resource, and it's Valemont:** Google accounts, Meta portfolio, Vercel account, Wise, Stripe. Turn on two-factor authentication everywhere.
- **Backups:** the downloadable JSON copy contains personal data, keep it somewhere private. A restore always goes into a **new** Sheet, never on top of production.

Delays to plan for (summary)

What	Delay	Meanwhile
Google verification for `gmail.send` (External app in production)	several days to a few weeks	Internal app (Workspace), or Testing mode with your two addresses, or pre-filled Gmail draft
Meta App Review (Instagram comments/messages)	a few days to a few weeks	Make "Watch comments" relay to the inbox
LinkedIn Community Management API	long, not guaranteed	.xls/.xlsx exports imported every week
Vercel domain certificate	a few minutes to a few hours (DNS)	`*.vercel.app` address for testing (remember to update `APP_BASE_URL` and the Google URIs afterwards)